

6710405389 ธนัต เทพโพธา

Answer Question

1.โปรแกรมอะไร

ตอบ โปรแกรม เรียกรถแท็กซี่ สำหรับคนขับรถ และผู้โดยสาร (คล้าย Grab/Bolt)

2.ใช้สำหรับทำอะไร application ที่พัฒนามีคุณลักษณะ (characteristics) อย่างไร

ตอบ ใช้สำหรับเรียกรถโดยสาร เป็น application ที่ต้องการลักษณะ ทั้งแบบ request และ response รวมถึง ต้องการการสื่อสารแบบ one way (การแจ้งอัปเดต location ของ driver)

3. ต้องการใช้ service model ใดของ Transport Layer (UDP หรือ TCP) เพราะอะไร

ตอบ ต้องการทั้ง 2 protocol ทั้ง UDP และ TCP โดยใช้

TCP: สำหรับสื่อสารแบบ request / response เพื่อตกลงข้อมูล เรียกรถ หรือการทำ operation ต่างๆ

UDP: สำหรับสื่อสารทางเดียว เพื่ออัปเดตตำแหน่ง Driver เพื่อความรวดเร็ว และไม่ทำให้ overload server

4.ออกแบบ Application-Layer Protocol ที่ประกอบด้วย request and response messages ที่ใช้ในการสื่อสารระหว่าง Client และ Server

ตอบ RTTP (Realtime Taxi/Driver Transport Protocol) โพรโตคอลสำหรับจัดการระบบ Taxi และ Passenger โดยรองรับการสื่อสารทั้ง TCP และ UDP รายละเอียดอยู่ที่เอกสารด้านล่าง

Version: 1.0

Status: Draft

Transport: TCP

Protocol Type: Application-layer communication protocol

Primary Use Case: Real-time communication between a taxi/ride-hailing server, drivers, and passengers

Introduction

RTP (Real-Time Taxi Transport Protocol) is a application-layer protocol designed for real-time communication between a taxi service server and its passengers.

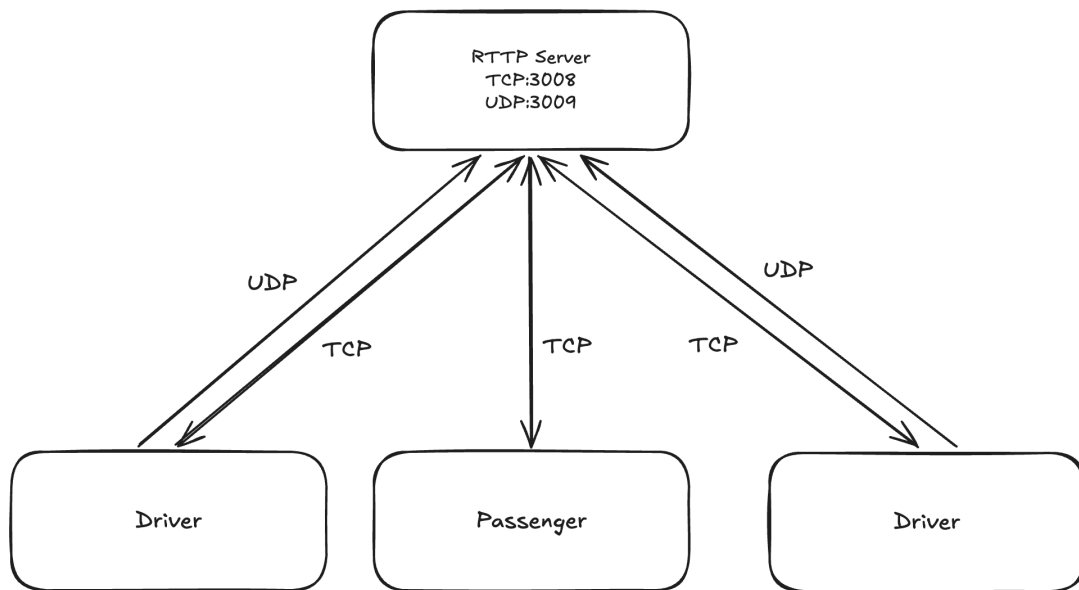
The protocol is designed to support communication between three roles:

- Server — manages drivers, passengers, and ride requests.
- Driver — registers with the server and provides driver-related information.
- Passenger — registers with the server and requests rides.

RTP operates over a persistent TCP connection. Messages are exchanged using a human-readable text-based format.

Architect

RTTP uses a client-server architecture. Three processes talk to each other: one Server, one or more Driver clients, one or more Passenger clients.



The server listens on a TCP port / UDP port and accepts multiple client connections. Each TCP connection represents one logical RTTP connection. The server therefore uses the connection itself to maintain the state and identity of the connected client.

Transport Layer

RTTP uses TCP and UDP as its transport protocol. Use TCP as a command and communication between server and its client. Use UDP specifically for communicate with driver to get driver location

The default development TCP server configuration is: localhost:3008

The default development UDP server configuration is: localhost:3009

Message Structure

An RTTP message consists of:

1. Start marker: Indicate start of the message
2. Protocol version: Indicate protocol version
3. Request Id: To determine which ACKNed message is corresponding to
4. Role: Role of this connection (Server,passenger,driver)
5. Id: Indicate connection identity
6. Type: Type of the message (INFORM/ACKN)
7. Status: Aced status
8. Operation: Operation of the message
9. payload: Additional data to fulfill operation
10. End marker: Indicate end of the message

use “\r\n\r\n” as delimiter

Example 1	Example 2
<RTTP_START> RTTP/1.0\r\n\r\n TYPE: INFORM\r\n\r\n OPERATION: REGISTER_DRIVER\r\n\r\n NAME: asdsd\r\n\r\n <RTTP_END>	<RTTP_START> RTTP/1.0\r\n\r\n TYPE: ACKN\r\n\r\n OPERATION: REGISTER_DRIVER\r\n\r\n DRIVERID: D001\r\n\r\n <RTTP_END>

Message Types

Type	Description
INFORM	Initiates something or notifies one-way. awaited via connection.inform(), which resolves when a matching ACKN with the same REQUEST_ID arrives.
ACKN	A reply to a prior INFORM (or a server-initiated notice like ESTAB). Always carries STATUS. Sent with connection.ackn(obj, requestId), echoing the original REQUEST_ID

INFORM is used when a client or server sends information or initiates an operation.

Example:

TYPE: INFORM

OPERATION: REGISTER_PASSENGER

ACKN is used to acknowledge or respond to an previous INFORM operation.Will have the same operation as INFORM

Example:

TYPE: ACKN

STATUS: 201

OPERATION: REGISTER_PASSENGER

Role

role is used to identify the role of the connection and will be send with every message

Role	Used By	Id meaning
Server	Default when server is created	always "0"
Driver	A connection after REGISTER_DRIVER	server-assigned driver id (e.g. D001)
Passenger	A connection after REGISTER_PASSENGER	server-assigned passenger id (e.g. P001)
Unknown	Default before registration completes	null

Status

status is included in every ACKedF message to have more control about the result of the operation. RTTP uses HTTP-like status code for convenience.

Example status are:

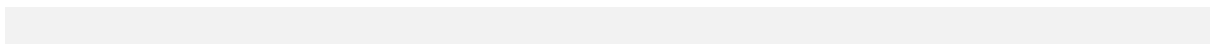
Status	Meaning
200	Successful request
201	Created
401	Unauthorized
500	Internal Server Error

Connection Identity

Each RTTP connection maintains two identity concepts:

1. Local Identity
2. Remote Identity

each identity will have role and id



```

interface ConnectionIdentity {
    role: ConnectionRole; // SERVER | DRIVER | PASSENGER | UNKNOWN
    id: string | null;      // e.g. driver id, passenger id, "0" for server
}

```

Local Identity

The local identity represents the current connection from the perspective of the local endpoint.

For example:

Local Identity:

```

role = PASSENGER
id = P001

```

Remote Identity

The remote identity represents the entity on the other side of the connection.

For example:

Remote Identity:

```

role = SERVER
id = 0

```

Identity may be assigned after the REGISTER_PASSENGER / REGISTER_DRIVER is operated.

Operation

This table below show all operation available in RTTP protocol

Operation	Type	Payload
ESTAB (Establish)	ACKN	{}

REGISTER_DRIVER	INFORM	{name}
	ACKN	{driverid}
REGISTER_PASSENGER	INFORM	{name}
	ACKN	{passengerid}
REQUEST_RIDE	INFORM	{pickuplat,pickuplng,destinationlat,destinationlng}
	ACKN	{driverid,driverlat,driverlng}
OFFER_RIDE	INFORM	{pickuplat,pickuplng,destinationlat,destinationlng}
	ACKN	{driverid,driverlat,driverlng}
ACCEPT_RIDE	INFORM	{}
	ACKN	{}
START_LOCATION	INFORM	{}
	ACKN	{locationtoken}
LOCATION_REPORT	INFORM	{locationtoken,driverid,driverlat,driverlng}
STOP_LOCATION	INFORM	{locationtoken}
	ACKN	{}
END_RIDE	INFORM	{}
	ACKN	{}
FIN	INFORM	{}
	ACKN	{}
NOT_IMPLEMENTED	INFORM	{}
	ACKN	{}
ERROR	INFORM	{}
	ACKN	{}

ESTAB

Sent automatically by the server the instant a TCP socket opens, before the client sends anything. Confirms the connection and gives the client its initial SERVER role/id context.

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: 0
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 200
OPERATION: ESTAB
<RTTP_END>
```

REGISTER_DRIVER

A newly-connected client registers itself as a driver. The server assigns a driver id and set the connection's remote identity to DRIVER.

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: a1f2...
ROLE: UNKNOWN
```



```
ID: N/A
TYPE: INFORM
OPERATION: REGISTER_DRIVER
NAME: Driver1
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: a1f2...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 201
OPERATION: REGISTER_DRIVER
DRIVERID: D001
<RTTP_END>
```

REGISTER_PASSENGER

Same idea as REGISTER_DRIVER, for a passenger connection. The server assigns a passenger id and role.

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: b7c9...
ROLE: UNKNOWN
ID: N/A
```

```
TYPE: INFORM
OPERATION: REGISTER_PASSENGER
NAME: Passenger1
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: b7c9...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 201
OPERATION: REGISTER_PASSENGER
PASSENGERID: P001
<RTTP_END>
```

REQUEST_RIDE

Passenger asks the server to find a driver. The server waits for a connected driver, ACKs the passenger with that driver's id and current location, and separately INFORMs the chosen driver via OFFER_RIDE.

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: c001...
ROLE: PASSENGER
```

```
ID: P001
TYPE: INFORM
OPERATION: REQUEST_RIDE
PICKUPLAT: 40.7128
PICKUPLNG: -74.0060
DESTINATIONLAT: 34.0522
DESTINATIONLNG: -118.2437
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: c001...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 201
OPERATION: REQUEST_RIDE
DRIVERID: D001
DRIVERLAT: 40.7128
DRIVERLNG: -74.0060
<RTTP_END>
```

OFFER_RIDE

Server notifies drivers (depends on application implementation) that a ride is available. The driver ACKs to accept

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: d222...
ROLE: SERVER
ID: 0
TYPE: INFORM
```

```
OPERATION: OFFER_RIDE
PICKUPLAT: 40.7128
PICKUPLNG: -74.0060
DESTINATIONLAT: 34.0522
DESTINATIONLNG: -118.2437
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: d222...
ROLE: DRIVER
ID: D001
TYPE: ACKN
STATUS: 200
OPERATION: OFFER_RIDE
DRIVERID: D001
DRIVERLAT: 40.7128
DRIVERLNG: -74.0060
<RTTP_END>
```

ACCEPT_RIDE

Once the driver's ACKN for OFFER_RIDE comes back, the server INFORMs the driver's connection that a previous ACKNed offer ride has accepted (Not taken by other drivers).

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: e333...
ROLE: SERVER
ID: 0
TYPE: INFORM
```

```
OPERATION: ACCEPT_RIDE
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: e333...
ROLE: DRIVER
ID: D001
TYPE: ACKN
OPERATION: ACCEPT_RIDE
<RTTP_END>
```

START_LOCATION

Driver requests a short-lived location token before it starts streaming GPS updates over UDP. The token is used to authorize the REPORT_LOCATION stream that follows.

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: f444...
ROLE: DRIVER
ID: D001
TYPE: INFORM
```

OPERATION: START_LOCATION

<RTTP_END>

Example response (ACKN)

<RTTP_START>

RTTP/1.0

REQUEST_ID: f444...

ROLE: SERVER

ID: 0

TYPE: ACKN

STATUS: 200

OPERATION: START_LOCATION

LOCATIONTOKEN: abc123

<RTTP_END>

REPORT_LOCATION (UDP)

Fired repeatedly (e.g. every second) directly to the UDP location server on port 3309, outside the TCP request/response cycle, so location traffic never queues behind messages. No ACKN it is fire-and-forget

Example message

<RTTP_START>

RTTP/1.0

REQUEST_ID: g555...

ROLE: DRIVER

ID: D001

TYPE: INFORM

```
OPERATION: REPORT_LOCATION
DRIVERID: D001
DRIVERLAT: 40.712900
DRIVERLNG: -74.005900
LOCATIONTOKEN: abc123
<RTTP_END>
```

END_LOCATION

Driver tells the server to close out the location-token session once the ride is finishing, stopping the REPORT_LOCATION stream.

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: h666...
ROLE: DRIVER
ID: D001
TYPE: INFORM
OPERATION: END_LOCATION
LOCATIONTOKEN: abc123
<RTTP_END>
```

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: h666...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 200
OPERATION: END_LOCATION
<RTTP_END>
```

END_RIDE

Driver signals the ride is complete. The server ACKs the driver directly, then separately looks up the matching passenger connection and INFORMs it, which the passenger ACKs in turn. Driver → Server → Passenger

Example message

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: i777...
ROLE: DRIVER
ID: D001
TYPE: INFORM
OPERATION: END_RIDE
```

<RTTP_END>

Example response (ACKN)

<RTTP_START>
RTTP/1.0
REQUEST_ID: i777...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 200
OPERATION: END_RIDE
<RTTP_END>

FIN

Graceful shutdown from either side. The server ACKs, then the connection is closed on the server side.

Example message

<RTTP_START>
RTTP/1.0
REQUEST_ID: j888...
ROLE: DRIVER
ID: D001
TYPE: INFORM
OPERATION: FIN
<RTTP_END>

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: j888...
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 200
OPERATION: FIN
<RTTP_END>
```

NOT_IMPLEMENTED

This operation is send as ACKN if the INFORM operation is not implemented or has handle event on a application level

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: 0
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 400
OPERATION: NOT_IMPLEMENTED
<RTTP_END>
```

ERROR

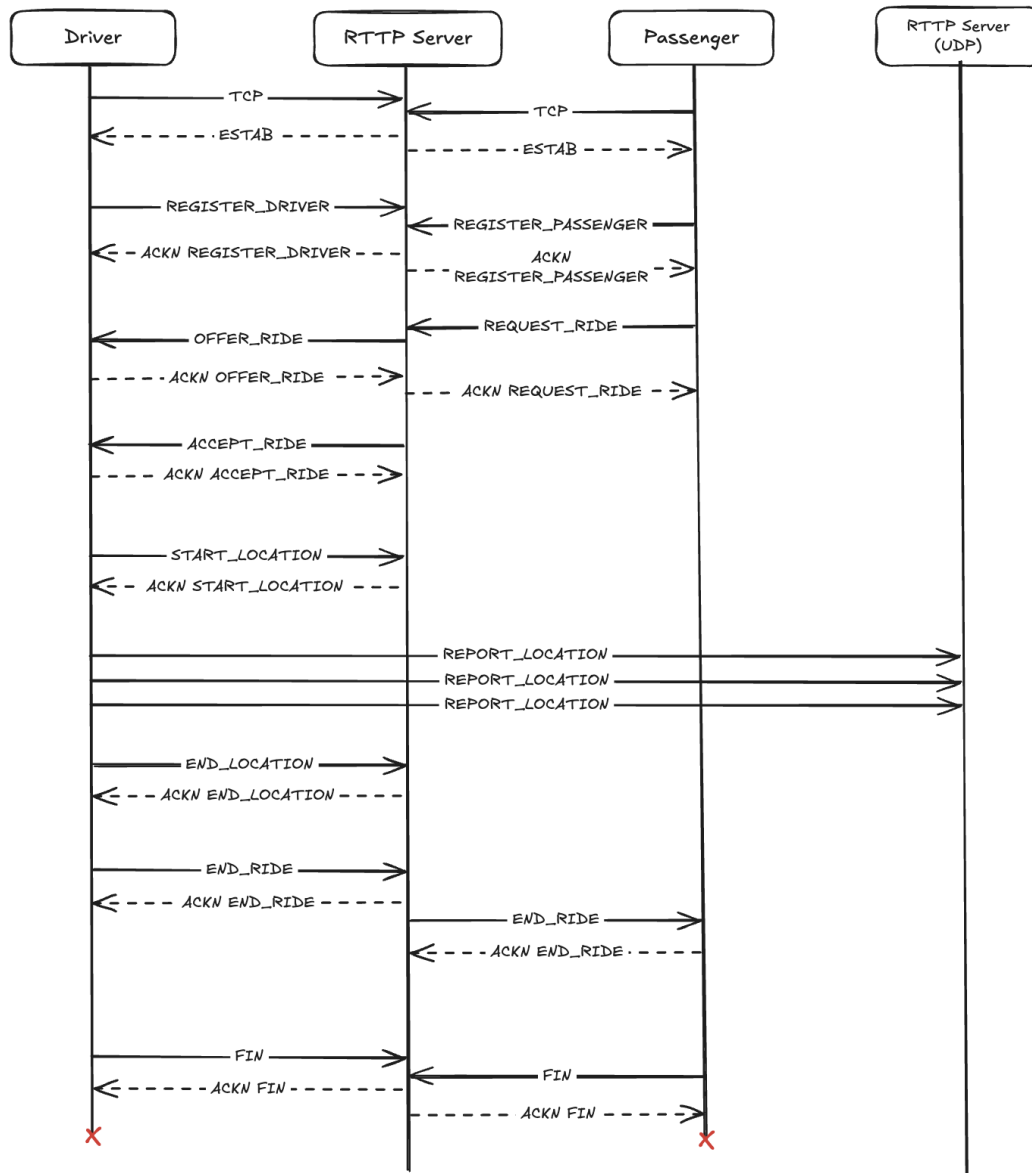
Same as NOT_IMPLEMENTED this operation is sent when error is occurred on server side

Example response (ACKN)

```
<RTTP_START>
RTTP/1.0
REQUEST_ID: 0
ROLE: SERVER
ID: 0
TYPE: ACKN
STATUS: 400
OPERATION: ERROR
MESSAGE: Invalid operation
```

<RTTP_END>

Operation Flow



1. both driver and passenger need to be registered to the server
2. Passenger REQUEST_RIDE and server offer ride to selected driver
3. driver that ACKN to that offer will be the driver of that ride and server notify passenger with ACKN on previous REQUEST_RIDE
4. Server notify driver again with ACCEPT_RIDE to confirm to the driver that the offer is valid (not taken by other drivers)
5. Driver send START_LOCATION to tell server that will send location report (to get location token for security reason)
6. Driver send REPORT_LOCATION to Server using UDP (port 3009)
7. When ride is about to end driver send END_LOCATION to invalidate that location token
8. Driver send END_RIDE to end the ride
9. Server notify passenger to end ride
10. Driver / Passenger send FIN to the server to close connection

Code Example

Server - Register handler

RTTPServer.on(operation, handler) maps one OPERATION to one async handler. replying NOT_IMPLEMENTED if operation is missing

```
server.on(RTTPOperation.REGISTER_DRIVER, (connection, request) => {  
  //server register driver  
  connection.ackn(  
    {  
      operation: RTTPOperation.REGISTER_DRIVER,  
      type: RTTPType.ACKN,  
      status: 201,  
      payload: { driverid: driverId },  
    },  
    request.requestid,  
  );  
});  
  
server.on(RTTPOperation.REGISTER_PASSENGER, (connection, request) => {  
  //server register passenger  
  connection.ackn(  
    {  
      operation: RTTPOperation.REGISTER_PASSENGER,  
      type: RTTPType.ACKN,  
      status: 201,  
      payload: { passengerid: passengerId },  
    },  
    request.requestid,  
  );  
});
```

Client - sending request and awaited for reply

connection.inform() sends an INFORM, stores a pending Promise keyed by REQUEST_ID, and resolves it when the matching ACKN is decoded in receive():

```

const requestRide = await client.inform({
  type: RTTPTType.INFORM,
  operation: RTTPOperation.REQUEST_RIDE,
  payload: {
    pickuplat: "40.7128",
    pickuplng: "-74.0060",
    destinationlat: "34.0522",
    destinationlng: "-118.2437",
  },
});

console.log(
  `Ride request acknowledged with driver ID: ${requestRide.payload.driverid}`,
);

```

Driver - streaming location over UDP

reportLocation() is fire-and-forget: it does not go through inform(), sends over a UDP socket to the location server.

```

while (rideActive) {
  await driver.reportLocation(
    { lat: lat.toFixed(6), lng: lng.toFixed(6) },
    start.payload.locationtoken,
  );
  lat += 0.0001;
  lng += 0.0001;
  await Bun.sleep(1000);
}

```

Server - receive location update over UDP

server.onLocationMessage() is used to capture any incoming location from UDP. Trigger one async handler and not reply to the message

```

server.onLocationMessage((message, port, addr) => {
  Logger.log(message);
});

```